

Catálogo de Eventos del Ecosistema

Documento normativo. Este catálogo es la **única fuente de verdad** sobre los nombres, productores, consumidores y estructura de los eventos del ecosistema. El enunciado del proyecto final y los talleres (retos) lo referencian; **ninguno de ellos redefine eventos por su cuenta**. Si un reto necesita un evento nuevo, se agrega aquí primero.

1. Convención de nomenclatura

<entidad>.<acción-en-pasado>

- **Minúsculas**, separadas por punto. Sin acentos, sin guiones, sin `camelCase`.
- El verbo va **siempre en pasado**: el evento describe algo que **ya ocurrió** y por lo tanto es un hecho inmutable. No existen eventos como `crear.empleado` o `empleado.crear`.
- La entidad es el sustantivo del dominio que cambió de estado (`empleado` , `usuario` , `cuenta` , `vacaciones`).

Nota sobre el Reto 00 (DDD). En el modelado conceptual del Event Storming los eventos se escriben en `PascalCase` (`EmpleadoCreado`), que es la convención de DDD para *domain events*. Al implementarlos como mensajes en el broker se traducen a esta convención de canal: `EmpleadoCreado` → `empleado.creado`. Son el mismo evento en dos capas distintas.

2. Sobre técnico del mensaje (envelope)

Todo evento publicado al broker debe ir envuelto en la siguiente estructura. Los datos propios del evento van dentro de `data`; el resto son metadatos comunes a todos los eventos.

```

{
  "id": "3f9b2c10-8e4a-4d6b-9f21-7c0a5d8e1234",
  "type": "empleado.creado",
  "version": 1,
  "occurredAt": "2026-03-01T14:32:05Z",
  "producer": "empleados-service",
  "data": {
    "...": "carga útil específica del evento"
  }
}

```

Campo	Obligatorio	Descripción
id	✓	Identificador único del mensaje (UUID). No es el id de la entidad.
type	✓	Nombre del evento, tal como aparece en la tabla de la sección 3.
version	✓	Versión del esquema del evento. Empieza en 1 .
occurredAt	✓	Instante en que ocurrió el hecho, en UTC (ISO-8601).
producer	✓	Nombre del microservicio que lo publicó.
data	✓	Carga útil. Su estructura se define por evento en la sección 3.

2.1 Deduplicación

El campo `id` existe para que **todo consumidor pueda tolerar recibir el mismo evento más de una**

vez. Un broker puede reentregar un mensaje si el consumidor se cae antes de confirmarlo, y el propio productor puede reintentar una publicación fallida.

Regla mínima exigida desde el **Reto 04**: antes de procesar un evento, el consumidor verifica si ya procesó ese `id` ; si es así, lo descarta y confirma el mensaje sin repetir el efecto. Una tabla `eventos_procesados(id, procesadoEn)` es suficiente en esta etapa.

El mecanismo formal (Inbox Pattern, DLQ, reintento con backoff exponencial) se estudia en el **Reto 29 – Idempotencia y Mensajería Confiable**. Lo que se pide aquí es la versión mínima que evita el bug más común del proyecto: correos de bienvenida duplicados y cuentas reactivadas indebidamente.

2.2 Datos sensibles

Ningún evento transporta contraseñas, ni en texto plano ni hasheadas. Cuando hay que transmitir un credencial de un solo uso (activación o recuperación de contraseña), se transporta un **token con expiración**, nunca la contraseña. Ver `usuario.creado` y `usuario.recuperacion`.

3. Catálogo

Evento	Productor	Consumidores	Se introduce en
<code>empleado.creado</code>	<code>empleados-service</code>	<code>auth-service</code> , <code>perfiles-service</code> , <code>notificaciones-service</code>	Reto 04
<code>empleado.actualizado</code>	<code>empleados-service</code>	<code>perfiles-service</code>	Reto 04
<code>empleado.retirado</code>	<code>empleados-service</code>	<code>auth-service</code> , <code>perfiles-service</code> , <code>notificaciones-service</code>	Reto 04
<code>usuario.creado</code>	<code>auth-service</code>	<code>notificaciones-service</code>	Reto 05
<code>usuario.recuperacion</code>	<code>auth-service</code>	<code>notificaciones-service</code>	Reto 05
<code>cuenta.activada</code>	<code>auth-service</code>	<code>notificaciones-service</code>	Reto 05
<code>cuenta.desactivada</code>	<code>auth-service</code>	<code>notificaciones-service</code>	Reto 05
<code>vacaciones.programadas</code>	<code>vacaciones-service</code>	<code>notificaciones-service</code>	Reto 04
<code>vacaciones.iniciadas</code>	<code>vacaciones-service</code>	<code>auth-service</code> , <code>notificaciones-service</code>	Reto 05
<code>vacaciones.finalizadas</code>	<code>vacaciones-service</code>	<code>auth-service</code> , <code>notificaciones-service</code>	Reto 05

3.1 empleado.creado

Se publica cuando RRHH registra un nuevo empleado y la operación se persistió con éxito.

```
{
  "empleadoId": "E001",
  "nombre": "Juan",
  "apellido": "Pérez",
  "email": "juan.perez@empresa.com",
  "numeroEmpleado": "EMP-2026-001",
  "cargo": "Desarrollador Senior",
  "area": "Tecnología",
  "departamentoId": "IT",
  "fechaIngreso": "2026-02-10",
  "estado": "ACTIVO"
}
```

Reacciones esperadas:

Consumidor	Acción
auth-service	Crea la cuenta en estado inactivo , sin contraseña válida. Genera un token de activación y publica <code>usuario.creado</code> .
perfiles-service	Crea un perfil por defecto con los datos básicos del evento.
notificaciones-service	Registra la traza del alta. El correo de bienvenida no se dispara aquí, sino con <code>usuario.creado</code> (que es el que trae el token).

3.2 empleado.actualizado

Se publica cuando cambia información del empleado que otros contextos replican.

```
{
  "empleadoId": "E001",
  "nombre": "Juan",
  "apellido": "Pérez Gómez",
  "email": "juan.perez@empresa.com",
  "cargo": "Tech Lead",
  "area": "Tecnología",
  "departamentoId": "IT"
}
```

Reacciones esperadas: `perfiles-service` sincroniza los campos replicados en el perfil.

3.3 empleado.retirado

Se publica cuando RRHH marca al empleado como `RETIRADO` . **El empleado no se borra físicamente:** cambia de estado y se conserva para auditoría.

```
{
  "empleadoId": "E001",
  "email": "juan.perez@empresa.com",
  "fechaRetiro": "2026-11-30T16:45:00Z",
  "motivo": "RENUNCIA"
}
```

Reacciones esperadas:

Consumidor	Acción
<code>auth-service</code>	Desactiva la cuenta de forma permanente y publica <code>cuenta.desactivada</code> con motivo: <code>"RETIRO"</code> .
<code>perfiles-service</code>	Archiva el perfil (no lo borra).
<code>notificaciones-service</code>	Registra la traza. El correo de despedida se dispara con <code>cuenta.desactivada</code> .

Nota: este evento sustituye al `empleado.eliminado` que aparecía en versiones anteriores del material. El nombre anterior sugería borrado físico, que contradice la regla de dominio establecida en el Reto 00 ("*Eliminar = marcar como RETIRADO, no se borra físicamente*").

3.4 usuario.creado

Se publica cuando el `auth-service` termina de crear la cuenta a raíz de `empleado.creado` . Es el evento que dispara el correo de bienvenida.

```
{
  "empleadoId": "E001",
  "email": "juan.perez@empresa.com",
  "tokenActivacion": "a3f8b2c9-1e4d-07f6-50a1-823bc79e14d5",
  "expiraEn": "2026-02-10T15:30:00Z"
}
```

 **Nunca** se incluye una contraseña, ni temporal ni generada. El empleado establece su propia contraseña usando el token. Ver `POST /auth/reset-password` en el Reto 05.

Reacciones esperadas: `notificaciones-service` envía el correo de bienvenida con el enlace o token de activación.

3.5 usuario.recuperacion

Se publica cuando un usuario solicita recuperar su contraseña vía `POST /auth/recover-password` .

```
{
  "email": "juan.perez@empresa.com",
  "tokenRecuperacion": "9c1d4e7a-2b8f-4a30-b6e5-1f207d9a4c88",
  "expiraEn": "2026-05-14T11:00:00Z"
}
```

Reacciones esperadas: `notificaciones-service` envía el correo de recuperación.

3.6 cuenta.activada

Se publica cada vez que una cuenta pasa a estado habilitado: al establecer la contraseña por primera vez, o al regresar de vacaciones.

```
{
  "empleadoId": "E001",
  "email": "juan.perez@empresa.com",
  "motivo": "ACTIVACION_INICIAL"
}
```

motivo	Cuándo
ACTIVACION_INICIAL	El empleado estableció su contraseña con el token de activación.
FIN_VACACIONES	Reactivación automática al consumir <code>vacaciones.finalizadas</code> .

3.7 cuenta.desactivada

Se publica cada vez que una cuenta deja de poder autenticarse. El campo `motivo` distingue si la desactivación es temporal o permanente; **no existe un evento separado para el caso permanente.**

```
{
  "empleadoId": "E001",
  "email": "juan.perez@empresa.com",
  "motivo": "VACACIONES",
  "permanente": false
}
```

motivo	permanente	Cuándo
VACACIONES	false	Al consumir <code>vacaciones.iniciadas</code> .
RETIRO	true	Al consumir <code>empleado.retirado</code> .

3.8 vacaciones.programadas

Se publica cuando RRHH registra un período de vacaciones que aún **no ha comenzado.**

```
{
  "vacacionesId": "V-2026-0042",
  "empleadoId": "E001",
  "email": "juan.perez@empresa.com",
  "fechaInicio": "2026-03-15",
  "fechaFin": "2026-03-30",
  "diasHabiles": 12
}
```

Reacciones esperadas: `notificaciones-service` envía la confirmación del período al empleado.

Este evento **no** desactiva la cuenta. La desactivación ocurre en `fechaInicio` , con `vacaciones.iniciadas` .

3.9 `vacaciones.iniciadas`

Lo publica el **scheduler** del `vacaciones-service` al llegar la `fechaInicio` del período.

```
{
  "vacacionesId": "V-2026-0042",
  "empleadoId": "E001",
  "email": "juan.perez@empresa.com",
  "fechaInicio": "2026-03-15",
  "fechaFin": "2026-03-30"
}
```

Reacciones esperadas:

Consumidor	Acción
<code>auth-service</code>	Desactiva la cuenta temporalmente y publica <code>cuenta.desactivada</code> con motivo: "VACACIONES" .
<code>notificaciones-service</code>	Notifica al empleado el inicio de su período.

3.10 vacaciones.finalizadas

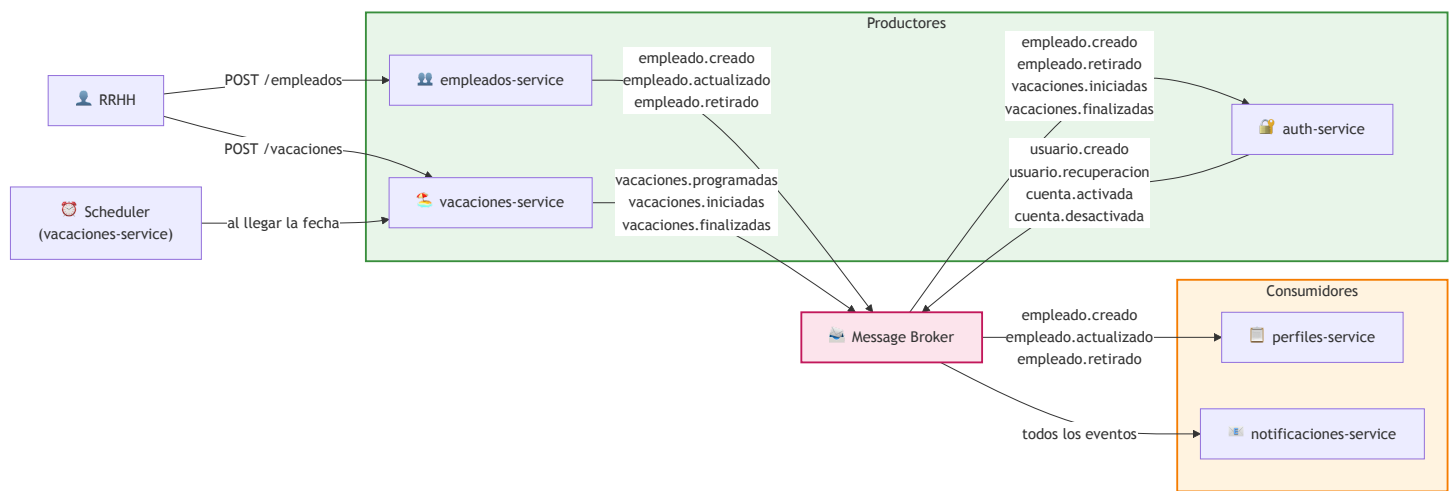
Lo publica el **scheduler** del `vacaciones-service` al llegar la `fechaFin` del período.

```
{
  "vacacionesId": "V-2026-0042",
  "empleadoId": "E001",
  "email": "juan.perez@empresa.com",
  "fechaFin": "2026-03-30"
}
```

Reacciones esperadas:

Consumidor	Acción
auth-service	Reactiva la cuenta solo si el empleado sigue en estado ACTIVO y publica <code>cuenta.activada</code> con motivo: <code>"FIN_VACACIONES"</code> . Si el empleado fue retirado durante sus vacaciones, la cuenta no se reactiva.
notificaciones-service	Envía la bienvenida de regreso.

4. Diagrama de flujo de eventos



5. Trazabilidad: reto ↔ eventos

Reto	Eventos que debe implementar
03 – Comunicación Asincrónica	<code>empleado.creado</code> , <code>empleado.actualizado</code> , <code>empleado.retirado</code> , <code>vacaciones.programadas</code>
04 – Seguridad JWT	<code>usuario.creado</code> , <code>usuario.recuperacion</code> , <code>cuenta.activada</code> , <code>cuenta.desactivada</code> , <code>vacaciones.iniciadas</code> , <code>vacaciones.finalizadas</code>
07 – Observabilidad	Todos: cada consumo de evento debe generar un log estructurado con <code>traceId</code> y el <code>id</code> del mensaje.
11 – Saga Pattern	Todos, más las transacciones compensatorias de cada uno.
21 – AsyncAPI	Especificación formal en YAML de al menos cuatro de estos canales.
28 – Idempotencia	Inbox Pattern y DLQ para todos los consumidores de esta tabla.

6. Historial de cambios del catálogo

Versión	Fecha	Cambio
1.0	2026-08-01	Versión inicial. Unifica la convención de nomenclatura, incorpora los eventos de vacaciones y los del <code>auth-service</code> , y sustituye <code>empleado.eliminado</code> por <code>empleado.retirado</code> y <code>cuenta.desactivada.permanente</code> por <code>cuenta.desactivada</code> con <code>motivo / permanente</code> .